

Repaso

Clase 15

¿Qué es un objeto?

Un objeto es una colección de datos (**atributos**) con un comportamiento (**métodos**) asociado en una única entidad.

Los objetos nos ayudan a crear tipos de **datos más complejos y representativos** a los que nos ofrecen los lenguajes de programación.

Los objetos interactúan entre sí mediante el envío de “mensajes”, estos conforman los elementos fundamentales para la POO.

Clases

Una clase describe las propiedades o atributos de objetos y las acciones o métodos que pueden hacer o ejecutar dichos objetos.

- **Cuando creamos un objeto, creamos una instancia de la clase.**
- Una instancia es un objeto individualizado por los valores que tomen sus atributos o propiedades.
- La interfaz pública del objeto está formada por las propiedades y métodos que otros objetos pueden usar para interactuar con él.

Sintaxis

```
# camel_case.py
```

```
class CamelCase:
```

```
    atributo_de_clase = "Es el mismo para toda instancia"
```

```
    # Constructor
```

```
    def __init__(self, atri1, atri2):
```

```
        self.atri1 = atri1
```

```
        self.atri2 = atri2
```

```
        self.atri3 = 0
```

```
    def metodo_de_instacia(self, par1):
```

```
        pass
```

```
    def __str__(self):
```

```
        return f"{atri1}, {atri2}"
```

```
# main.py
```

```
# Instancio un objeto y le envié un  
mensaje.
```

```
from camel_case import CamelCase  
obj1 = CamelCase('par1', 'par2')
```

```
obj1.metodo_de_instancia(par1)
```

```
print(obj1.atri2)
```

Atributos y métodos “no públicos” en Python (Encapsulamiento)

Al no existir encapsulamiento en Python, se ha acordado una convención entre los programadores. La cual consiste en anteponer el caracter `_` antes del nombre del atributo o método para informar que no se debería acceder a esta desde afuera de la clase. Aunque en la realidad el acceso aun sigue siendo posible.

La funcionalidad del doble guión bajo (name mangling) `__` es para que no haya conflictos de nombres con subclases aunque también crea un falso encapsulamiento como el simple guión bajo.

Ejemplo

```
class Test:
    def __init__(self, n1, n2, n3):
        self.n1 = n1
        self._n2 = n2 # privado
        self.__n3 = n3 # name mangling

    def _metodo_privado(self):
        print("Este es un método privado")

    def __metodo_privado2(self):
        print("Este es un método privado 2")
```

```
t = Test(11, 22, 33)
print(dir(t))

print(t.n1)
print(t.n2)
print(t.n3)
print(t._n2)
print(t.__n3)
print(t._Test__n3)
t.metodo_privado()
t._metodo_privado()
```

Herencia

- Una clase hija hereda los atributos y métodos de la clase padre.
- Una clase hija puede extender (agregar) sus propios atributos y métodos.
- Una clase hija puede sobrescribir los atributos y métodos de su clase base.

Herencia

class Empleado:

```
def __init__(self, nombre, dni, salario=1000):  
    self.nombre = nombre  
    self.salario = salario  
    self.dni = dni
```

```
def trabaja(self):  
    print("Estoy trabajando como empleado")
```

```
def __str__(self):  
    return f"Nombre: {self.nombre}"
```

class NoDocente(**Empleado**):

```
def __init__(self, nombre, dni, salario, cargo):  
    super().__init__(nombre, dni, salario)  
    self.cargo = cargo
```

class Docente(**Empleado**):

```
def __init__(self, nombre, dni, cargo):  
    super().__init__(nombre, dni)  
    self.cargo = cargo
```

```
def trabaja(self):  
    print("Estoy trabajando como Docente")
```


Polimorfismo (ejemplo)

```
class Vehiculo:
    def calcular_costo_seguro(self):
        pass

class Coche(Vehiculo):
    def calcular_costo_seguro(self):
        # Cálculo específico para coche
        return 1000

class Moto(Vehiculo):
    def calcular_costo_seguro(self):
        # Cálculo específico para moto
        return 800

class Bicicleta(Vehiculo):
    def calcular_costo_seguro(self):
        # Cálculo específico para bicicleta
        return 200
```

Uso del polimorfismo

```
vehiculos = [Coche(), Moto(), Bicicleta()]
```

```
# Sino implemente el metodo calcular_
for vehiculo in vehiculos:
```

```
    if isinstance(vehiculo, Coche):
        print(1000)
```

```
    elif isinstance(vehiculo, Moto):
        print(800)
```

```
    else:
        print(200)
```

Polimorfismo

```
for vehiculo in vehiculos:
    print(vehiculo.calcular_costo_seguro())
```

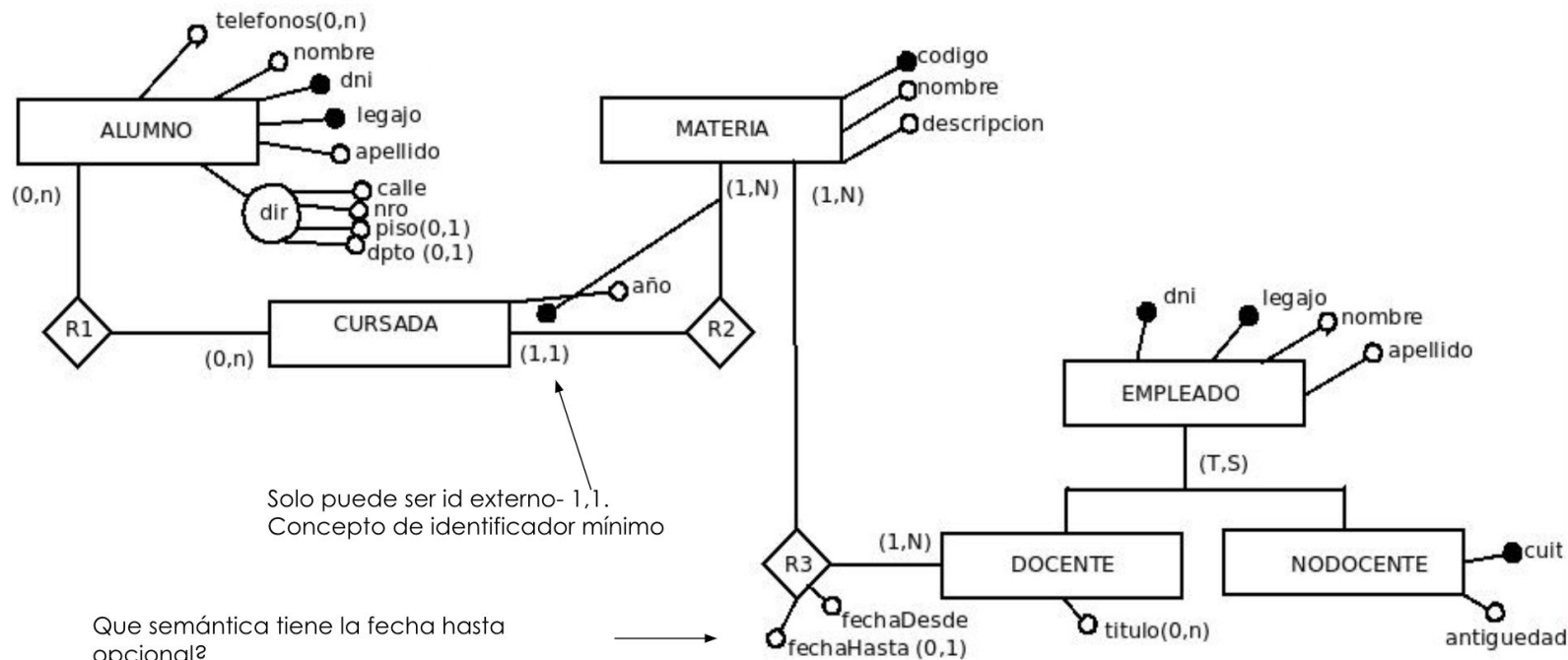
¿Qué es una Base de Datos?

- Es una colección de datos relacionados.
- Colección de archivos diseñados para servir a múltiples aplicaciones.
- Una BD es una colección coherente de datos con significados inherentes. Un conjunto aleatorio de datos no puede considerarse una BD. O sea los datos deben tener cierta lógica.
- Una BD se diseña, construye y completa de datos para un propósito específico. Está destinada a un grupo de usuarios concretos y tiene algunas aplicaciones preconcebidas en las cuales están interesados los usuarios.
- Una BD está sustentada físicamente en archivos en dispositivos de almacenamiento persistente de datos

Diseño Conceptual

El diseño conceptual parte de la especificación de requerimientos y su resultado es el esquema conceptual de la base de datos. Un esquema conceptual es una descripción de alto nivel de la estructura de la base de datos. El propósito del diseño conceptual es describir el contenido de información de la base de datos. Herramienta utilizada: Modelo Entidad-Interrelaciones (ER).

Resolución

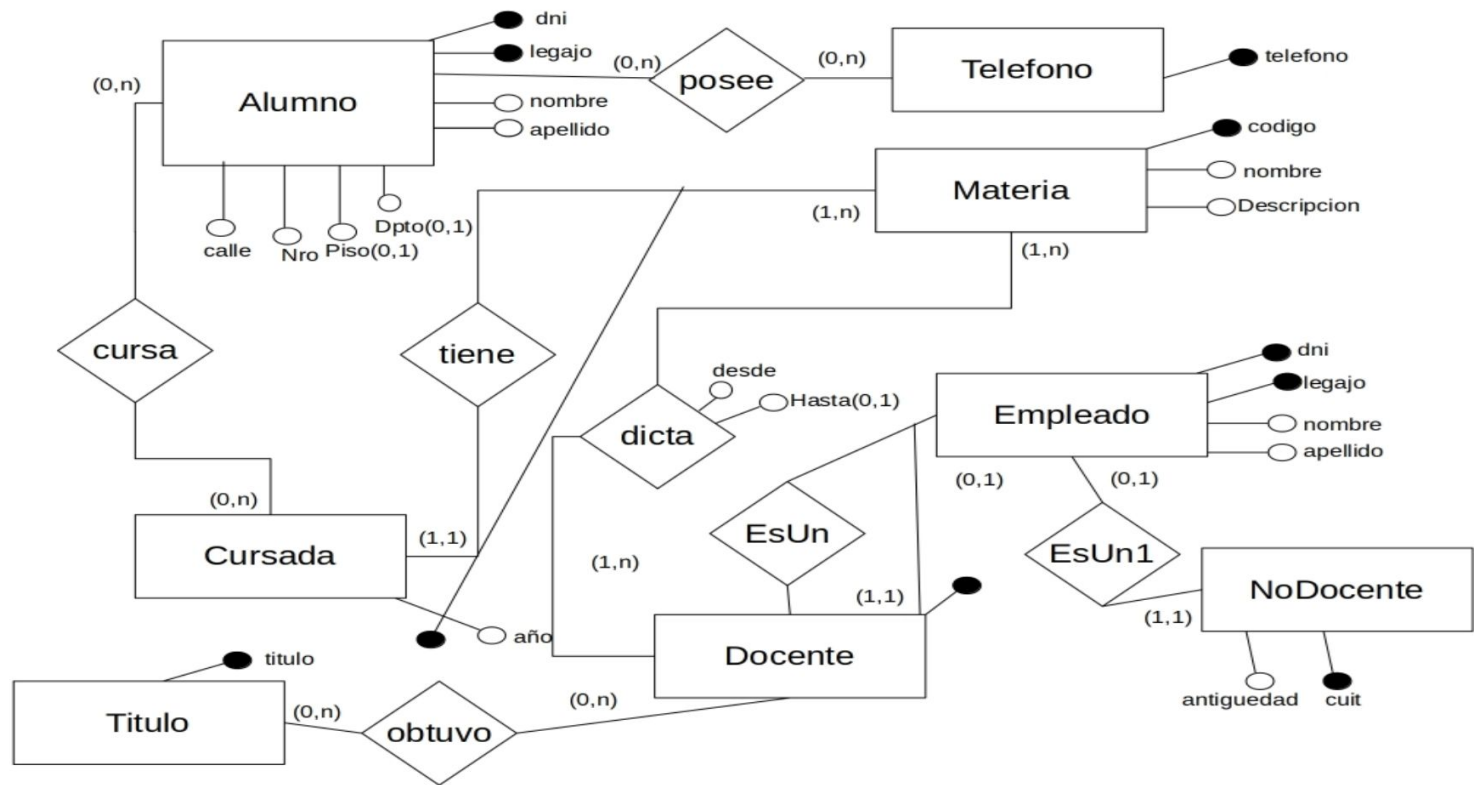


Modelado Lógico

El propósito de la generación de un modelo ER Lógico es convertir el esquema conceptual en un modelo más cercano a la representación entendible por el DBMS.

Recordemos que el diseño conceptual busca representar, de la forma más clara posible, las necesidades del usuario. Una vez cumplido este paso, el diseño lógico busca representar un esquema equivalente, que sea más eficiente para su utilización.

Modelo Lógico



Modelado Físico

El modelo relacional representa a una BD como una colección de archivos denominados **tablas**. Cada tabla se denomina relación y está integrada por **filas** y **columnas**. Cada fila se denomina **tupla** y cada columna representa un **atributo**.

Modelo Físico

Alumno = (dni, legajo, nombre, apellido, calle, nro, piso?, dpto?)

Telefono = (telefono)

Materia = (codigo, nombre, descripcion)

Cursada = (año, codigo(fk))

Empleado = (dni, legajo, nombre, apellido)

Docente = (dni(fk))

NoDocente = (cuit, dni(fk), antiguedad)

Titulo = (titulo)

obtuvo= (dni(fk), titulo(fk))

posee = (dni(fk), telefono(fk))

dicta = (codigo(fk), dni(fk), desde, hasta?)

curso = (dni(fk), año(fk), codigo(fk))

Componentes de un DBMS

- **DDL (data definition lenguaje):** Es el conjunto de comandos SQL utilizados para definir y modificar la estructura de la base de datos. Estos comandos se centran en la creación, modificación y eliminación de objetos de la base de datos, como tablas, vistas, índices, restricciones y procedimientos almacenados. Algunos ejemplos comunes de comandos DDL son CREATE, ALTER y DROP.
- **DML (Data Manipulation Language):** Se refiere al conjunto de comandos SQL utilizados para manipular y gestionar los datos almacenados en la base de datos. Estos comandos se utilizan para realizar operaciones como la inserción, actualización, eliminación y recuperación de datos de las tablas de la base de datos. Algunos ejemplos comunes de comandos DML son SELECT, INSERT, UPDATE y DELETE.

Comandos DDL

Comando	Descripción
CREATE	Crea una nueva tabla, base de datos o vista
ALTER	Modifica la estructura de una tabla existente
DROP	Elimina una tabla, base de datos o vista
TRUNCATE	Elimina todos los registros de una tabla
GRANT	Otorga permisos de acceso a usuarios y roles
REVOKE	Revoca los permisos de acceso previamente otorgados

Tipos de datos

- BOOLEAN (BOOL): 1, yes, y, t, true = true. 0, no, false, f = false.
- VARCHAR(n): Acepta como máximo n caracteres.
- TEXT: Cantidad teórica ilimitada de caracteres.
- SMALLINT: de -32,768 a 32,767.
- INT: de -2,147,483,648 to 2,147,483,647.
- SERIAL: Igual que int pero se generan automaticamente.
- float(n): como mínimo tendrá n decimales.
- NUMERIC(p, s): p es la cantidad total de digitos. s es la cantidad de decimales.
- DATE: 'yyyy-mm-dd'
- TIME: 'HH:MM', 'HH:MM:SS', 'HHMMSS'

DML

Comando	Descripción
SELECT	Recupera registros de una o varias tablas
INSERT	Inserta nuevos registros en una tabla
UPDATE	Actualiza los valores de los registros existentes
DELETE	Elimina registros de una tabla
MERGE	Combina operaciones INSERT, UPDATE y DELETE en una sola sentencia
COMMIT	Confirma las transacciones pendientes
ROLLBACK	Deshace las transacciones pendientes y restaura el estado anterior

Postgresql

```
1 import psycopg2
2
3 con = psycopg2.connect(database='postgres',
4                         user='postgres',
5                         password='cf1410')
6 cur = con.cursor()
7
8 comando = """CREATE TABLE persona (
9                 id serial PRIMARY KEY,
10                dni VARCHAR(8)
11                );"""
12
13 cur.execute(comando)
14 # print(cur.fetchone())
15
16 con.commit()
17 cur.close()
18 con.close()
```

Ventajas de numpy

NumPy es ampliamente utilizado en el procesamiento de datos, la ciencia de datos, el aprendizaje automático, la visualización de datos y otras áreas que involucran cálculos numéricos. Algunas de las ventajas clave de NumPy incluyen:

- **Eficiencia:** Los arreglos NumPy están optimizados para realizar operaciones numéricas rápidas y eficientes. Internamente, los arreglos NumPy están almacenados de manera contigua en la memoria, lo que permite realizar operaciones vectorizadas en lugar de bucles explícitos.
- **Funcionalidades matemáticas:** NumPy proporciona una amplia gama de funciones matemáticas y operaciones para realizar cálculos avanzados en arreglos. Estas funciones incluyen trigonometría, álgebra lineal, estadísticas, transformadas, funciones especiales y más.
- **Broadcasting:** NumPy permite realizar operaciones entre arreglos de diferentes formas y dimensiones mediante el broadcasting. Esto simplifica el proceso de realizar cálculos entre arreglos que no tienen las mismas dimensiones y permite escribir código más conciso.
- **Integración con otras bibliotecas:** NumPy es la base para muchas otras bibliotecas populares de Python utilizadas en ciencia de datos y aprendizaje automático, como Pandas, Matplotlib y Scikit-learn. Estas bibliotecas aprovechan las capacidades de NumPy para el manejo eficiente de datos y cálculos numéricos.

¿Qué es Pandas?

- Pandas es una biblioteca de análisis de datos de código abierto escrita en Python.
- Proporciona estructuras de datos de alto rendimiento y fáciles de usar, así como herramientas para manipular y analizar datos.
- Pandas es ampliamente utilizado en el campo de la ciencia de datos y se ha convertido en una herramienta esencial para cualquier persona que trabaje con datos en Python.

Matplotlib

- Matplotlib es una biblioteca popular en Python para crear gráficos y visualizaciones.
- Proporciona una amplia gama de opciones para representar datos en forma de gráficos de alta calidad.
- Es flexible y se puede utilizar en una variedad de aplicaciones, desde análisis de datos hasta generación de informes y presentaciones.



Ejercicio integrador

Resuelve el siguiente ejercicio usando python y postgres: Crear un programa en Python que brinde 3 opciones al usuario (iniciar sesion, crear cuenta y mostrar estadísticas). La info de cada usuario se debera guardar en una base de datos que contendra su email, sexo y contraseña. Para inciar sesión el usuario deberá ingresar su email y contraseña y si es correcta se imprimirán sus datos, caso contrario se le informará el error. La opción de crear cuenta cargará los datos y los guardará en una base de datos.

Por último, programar una función que muestre un gráfico torta que represente el porcentaje de usuarios segun su sexo.